



Government
of Canada

Gouvernement
du Canada

ESDC Development Community

Writing Good Change Logs

Le texte français est donné à la suite.

Background

Having a good changelog can be critical for those using your applications and services. It lets them know what has changed, and what they need to change. Ensuring it stays consistent and is clear helps those understand the changes better. Notes should be easy to read and convey a clear message to the reader.

General logs

The change log should be a list of changes made for each release of a product. The log should be stored in the root of the products sourcecode and called `CHANGELOG.md`.

All changes that are listed should indicate the *public* property, function, or feature name that is changing in clear and to-the-point detail providing a link (issue number, documentation, commit) for more details on the change. By *public* it mean the name that the users of the solution would know or recognize.

API logs

APIs should follow this logging template as they generally require strict versioning schemes to identify *breaking* changes. Semantic Versioning is the most common and recommended approach to versioning, so this template assumes that it is being used.

0.0.0 - Release Name

Breaking changes

- PropertyX in FunctionG now returns alphanumerics - fixes #00

New features and improvements

- FunctionT added for making streamlined calls to DB - completes #00

Fixes

- PropertyJ in FunctionB parses properly - fixes #00

Release Name 0.0.0 indicates the version being released. It could optionally include the - Release Name indicating a release name.

Breaking changes : This section lists all changes that require the implementer of your solution to make changes on their end to complete the update. If you are adding to this section you should be updating the major version number (**0.0.0**) in your version.

New features and improvements : This section lists all new features that have been added and any improvements made to existing features. Changes listed here should not cause the an implementer to make manual changes when updating; if they do it should be moved to *Breaking changes*. If you are adding to this section you should be updating the minor version number (0.**0**.0) in your version. If there are no new features, that should still be indicated in this section.

Fixes : This section lists any changes made to fix issues, improving back end operations, or patching other dependencies . Changes listed here should not cause the an implementer to make manual changes when updating; if they do it should be moved to *Breaking changes*. If you are adding to this section you should be updating the patch version number (0.0.**0**) in your version.

Application Logs

Application Logs are different from API logs because they speak to a different audience, the users of the application (not developers). They should avoid making references to details of code, like property names and instead make references to field names the user would see. Each listed change should be a link to documentation about the change. What you are describing in the change should be tailored to help *sell* the product.

Release Name - February 15, 2020

New Features

- You can subscribe to services now

Improvements

- You can now edit your user name

Fixes

- Links now have accessible contrast colour in dark mode

Operations

- Error tracking optimized for speed

The Top Header Release Name should be an identifiable name, a short description, for the release. It could optionally include the - February 15, 2020 (a date stamp) to indicate when the release was made.

New Features : This section includes new features added to the application. There should be a link to the detailed documentation of how the features works. *The whole list item could be the link.*

Improvements : The improvement section will tell users about the changes made to improve the existing features of the application. This section will help users understand why they should look for the new and improved

features.

Fixes : This section should detail which bugs have been fixed since the last release. This will inform users how the application should be performing in areas they know were broken. These fixes may not be part of the latest deployment, as they were patched previously, but should still be indicated here as part of the release.

Operations : These are any changes made related to the service of the application. These changes wouldn't impact users directly.

Texte français:

Une mise en page pour rédiger des journaux de modifications cohérents et informatifs, significatifs pour ceux qui les lisent.

Contexte

Avoir un bon journal des modifications (changelog) peut être essentiel pour les personnes utilisant vos applications et services. Il leur permet de savoir ce qui a changé et ce qu'elles doivent adapter. Veiller à ce qu'il reste cohérent et clair aide à mieux comprendre les changements apportés. Les notes doivent être faciles à lire et transmettre un message clair au lecteur.

Journaux généraux

Le journal des modifications doit être une liste des changements apportés à chaque version d'un produit. Le fichier doit être conservé à la racine du code source du produit et s'appeler `CHANGELOG.md`.

Tous les changements répertoriés doivent indiquer le nom de la propriété, fonction ou fonctionnalité *publique* modifiée de manière claire et concise, en fournissant un lien (numéro d'élément, documentation, commit) pour plus de détails sur le changement. Par *publique*, on entend le nom que les utilisateurs de la solution connaissent ou reconnaissent.

Journaux d'API

Les API devraient suivre ce modèle de journalisation, car elles requièrent généralement des schémas de gestion sémantique de version stricts pour identifier les changements majeurs ou incompatibles (*breaking changes*). La gestion sémantique de version (SemVer) est l'approche la plus courante et recommandée, ce modèle suppose donc son utilisation.

0.0.0 - Nom de la version

Changements incompatibles (Breaking changes)

- PropertyX dans FunctionG retourne maintenant des caractères alphanumériques - corrige #00

Nouvelles fonctionnalités et améliorations

- Ajout de FunctionT pour simplifier les appels vers la base de données - complète #00

Correctifs

- PropertyJ dans FunctionB est analysé correctement - corrige #00

Nom de la version 0.0.0 indique la version publiée. Il peut éventuellement inclure - Nom de la version pour préciser le nom de la version.

Changements incompatibles (Breaking changes) : Cette section répertorie tous les changements nécessitant une intervention de l'intégrateur de votre solution pour effectuer la mise à jour. Si vous ajoutez un élément à cette section, vous devez incrémenter le numéro de version majeure (**0.0.0**).

Nouvelles fonctionnalités et améliorations : Cette section répertorie toutes les nouvelles fonctionnalités ajoutées ainsi que les améliorations apportées aux fonctionnalités existantes. Les changements listés ici ne devraient pas obliger l'intégrateur à effectuer des modifications manuelles lors de la mise à jour; si c'est le cas, ils doivent être déplacés vers *Changements incompatibles*. Si vous ajoutez un élément à cette section, vous devez incrémenter le numéro de version mineure (0.**0**.0). S'il n'y a pas de nouvelle fonctionnalité, cela doit quand même être indiqué dans cette section.

Correctifs (Fixes) : Cette section répertorie les correctifs apportés à des bogues, les améliorations des opérations d'arrière-plan ou les mises à niveau de dépendances. Les changements indiqués ici ne doivent pas forcer l'intégrateur à des modifications manuelles; si c'est le cas, ils doivent être déplacés vers *Changements incompatibles*. Si vous ajoutez un élément à cette section, vous devez incrémenter le numéro de correctif (0.0.**0**).

Journaux d'application

Les journaux d'application diffèrent des journaux d'API car ils s'adressent à un public différent : les utilisateurs de l'application (et non les développeurs). Ils doivent éviter de faire référence à des détails de code

comme des noms de propriétés, et plutôt faire référence aux noms de champs visibles par l'utilisateur. Chaque changement listé devrait contenir un lien vers la documentation associée. La description doit être formulée de manière à valoriser et promouvoir le produit.

Nom de la version - 15 février 2020

Nouvelles fonctionnalités

- Vous pouvez désormais vous abonner aux services

Améliorations

- Vous pouvez désormais modifier votre nom d'utilisateur

Correctifs

- Les liens disposent désormais d'un contraste accessible en mode sombre

Opérations

- Optimisation du suivi des erreurs pour une vitesse accrue

L'en-tête Nom de la version doit être un nom identifiable, une courte description pour la version. Il peut éventuellement inclure - 15 février 2020 (un horodatage) pour indiquer la date de livraison.

Nouvelles fonctionnalités : Cette section comprend les nouvelles fonctionnalités ajoutées à l'application. Un lien vers la documentation détaillée expliquant le fonctionnement de la fonctionnalité devrait être fourni. *L'élément de liste en entier peut servir de lien.*

Améliorations : La section des améliorations informe les utilisateurs des améliorations apportées aux fonctionnalités existantes de l'application. Cette section aide les utilisateurs à comprendre pourquoi explorer les fonctionnalités optimisées.

Correctifs : Cette section détaille les anomalies corrigées depuis la version précédente. Elle indique aux utilisateurs le bon fonctionnement des éléments qui présentaient des défaillances. Ces correctifs peuvent avoir été déployés préalablement, mais doivent être mentionnés ici dans le cadre de la version.

Opérations : Il s'agit des changements relatifs au service et à l'infrastructure de l'application. Ces modifications n'ont pas d'impact direct sur les utilisateurs finaux.